Experiment 5 — Machine integers as a peer of machine reals

Mathilda — execution-speed experiments

Contents

Experiment 5 — Machine integers as a peer of machine reals

Dates: 2026-07-29 → 2026-07-30 · Commits: `b91cdcd` (compile), `86d75db` (narrowing kernels) · Code: `src/compile/`, `src/ndkernels.c`, `src/ndarray.c` · Result: the integer benchmarks stop being second-class — Game of Life 65.7 s → 260 ms

Common method in `README.md`.

Hypothesis

Everything in experiments 1–4 and 6 was built for `double`. `CT_INT` was in the compiler's type lattice from the start, but only `double` was a first-class citizen: 18 integer opcodes against 69 real ones, no integer buffers, no overflow discipline.

That is not a rounding error in coverage. A sieve, a Game of Life grid, a combinatorial search and `Range[n]` itself are all integer, and every one of them was falling off every fast path in the system at once.

The hypothesis was that integers had been treated as a variant of reals when they are a different type with a harder contract: reals may round, integers may not.

Why it is harder than it looks

A `double` that overflows becomes `Infinity` and everyone can see it. An `int64` that overflows wraps silently:

```
In[1]:= Compile[{{n, _Integer}}, n^3][3000000]
Out[1]= 8553255926290448384       (* was; the interpreter says 27000000000000000000 *)
```

Mathilda's contract is that a compiled body answers exactly as the interpreter does. An interpreter that promotes to GMP and a VM that wraps do not agree, so either the VM promotes (it cannot — no bigints in a register machine) or it detects and defers.

What was built

Overflow detection on every integer opcode that can overflow — `ADD_I`, `SUB_I`, `MUL_I`, `NEG_I`, `ABS_I`, `POWI_I`, `INC_I` and the immediate forms. On overflow the call is abandoned and the interpreter re-runs the body, which promotes to a bigint. The compiled answer is the interpreted one by construction. Loop accumulators inherit it: `Sum`, `Product` and a `Do` over an integer local lower to the same opcodes. Constant folding refuses to fold an overflowing integer operation for the same reason.

`RuntimeOptions -> {"CatchMachineIntegerOverflow" -> False}` (shorthand `RuntimeOptions -> "Speed"`) keeps the wrapped `int64`. It is opt-in because it makes the object answer differently from the interpreter once a result leaves machine range — this flag is semantics, not speed. The choice is baked into each instruction (`IF_NOCHK`) rather than tested per execution, so it costs nothing at run time; measured at 10^8 iterations, checks-on and checks-off are within noise of a build with no detection at all.

`NDT_INT64` buffers throughout the ND layer, with exact accumulation (`ci_add_i64`, `ci_mul_i64`) that abandons the whole result on the first overflow so the List path can answer with GMP. `Total[Range[10^6]^3]` still reaches GMP and is still exact.

Narrowing kernels (`86d75db`) — a new kernel category, real in, `NDT_INT64` out. `Floor`, `Ceiling`, `Round`, `IntegerPart` and `Sign` on a float64 buffer now produce the exact `Integers` the List path produces, instead of `1.0` where the List gives `1`. Before this they had to be excluded from the buffer path entirely.

Two crashes and a wrong answer, found on the way:

- `Compile[{{a,_Integer},{b,_Integer}}, Mod[a,b]][5, 0]` took the process down with `SIGFPE`. Both integer divisions now guard the two inputs the hardware traps on: a zero divisor, and `INT64_MIN / −1`, whose quotient is not representable.
- `Abs[-9223372036854775808]` returned a negative number — in the interpreter, not just under `Compile`. Negating `INT64_MIN` wraps to itself; it now promotes to a bigint like every other arithmetic head.

The exactness bug this exposed in packing

Automatic packing (experiment 6) means `Range[n]` is an `int64` buffer, so paths written when only `Compile[]` could make one now see them constantly. Sweeping all 79 registered kernel heads packed-against-plain found `Quotient` answering differently either side of the packing threshold:

```
Quotient[Range[1., 300.], 2]  ->  {0., 1., 1., 2.}   (packed)
Quotient[Range[1., 200.], 2]  ->  {0, 1, 1, 2}       (not packed)
```

The same expression, two answers, decided by length. That is the worst possible failure mode for an invisible representation, and it is only findable by a differential sweep over every registered head, not by testing the heads someone thought to test.

Results

Benchmark	Mathilda	Mathematica 14.0	NumPy / Python	vs WL	vs NumPy
Sieve of Eratosthenes to 10^7	664.56 ms	784.49 ms	26.44 ms	1.18x	1/25.14x
Collatz longest chain below 10^6	4.625 s	6.898 s	10.100 s	1.49x	2.18x
Game of Life, 256^2, 100 generations	260.53 ms	96.34 ms	92.90 ms	1/2.70x	1/2.80x
Naive recursive Fibonacci, fib(25)	365.11 ms	137.64 ms	12.19 ms	1/2.65x	1/29.96x

The sieve is the row that justifies the third column: 1.18× ahead of Mathematica and 25× behind NumPy. Against a competitor it looks fine; against the machine it is the worst integer row in the suite — and NumPy's version is a fair comparison, a vectorised slice assignment (`s[i*i::i] = False`) rather than a loop.

Collatz and `fib(25)` are CPython loops, not library calls: neither has a vectorised form. Mathilda beats CPython by 2.18× on Collatz and loses to it by 30× on `fib`, which is naive recursion — pure evaluator dispatch, no arrays — and is the one row in the whole suite where CPython beats both CAS.

The Game of Life row is the headline and it is not really about integers per se: the grid is integer, so a helper `f[x_] := body` binding it caused the packing gate to materialise 65536 nodes on every one of 100 generations. The DownValue exemption ("a rule that binds the whole value reads no element") had been written for float64 only, because a user symbol's `packed_int64_ok` is false. 65.7 s → 260 ms (253×), and it now agrees with Mathematica on the answer as well as being 2.9× rather than 722× off its time.

Still open

- Overflow defers to the interpreter rather than promoting in place, so a loop that overflows on its last iteration pays for the whole loop twice.
- `Im` remains excluded from the narrowing category: its kernel is a projection, not a narrowing.
- `ndt_get` is exact only to 2^{53}, so every int64 path must route through `ndt_get_i` or `memcpy`. That is a per-site discipline, not a type-system guarantee, and `make check-packed-aware` only checks opt-in, not correctness.

Why Mathilda is not the fastest here, and what it would take

row	Mathilda	best other	gap	cause
Sieve of Eratosthenes to 10^7	665 ms	26.4 ms (NumPy)	25.1×	a scalar strided write loop against a vectorise
Collatz longest chain	4.63 s	6.90 s (WL)	1.49× ahead	
Game of Life, 256^2, 100 gens	261 ms	92.9 ms (NumPy)	2.80×	eight separate rotations and an unfused sum
Naive recursive `fib(25)`	365 ms	12.2 ms (CPython)	30.0×	pattern-match dispatch per call

Three rows to explain, and they have three different causes.

The sieve is an algorithm difference, not an implementation one. NumPy's `s[i*i::i] = False` is a strided memset over a boolean array — one call into compiled C per prime. The compiled CAS form is a scalar `While` writing one element at a time. Both are the idiomatic spelling in their language.

The Game of Life is the unfused-composition gap: eight `RotateLefts`, a sum and four `UnitSteps` is thirteen passes over a 256^2 grid where a fused kernel makes two.

`fib(25)` is the evaluator itself — 243k calls, each a pattern match against three `DownValues`. This is the one row in the entire suite where CPython beats both CAS, and it is measuring rule dispatch rather than arithmetic.

The road to fastest

1. A strided fill/assign kernel. `s[[Range[a, n, d]]] = v` on a packed array should be a strided write loop in C, not a gather-scatter through a materialised position array. This is the same missing `start/step/n` selector as plan item 9.5, and it turns the sieve's inner statement into the same memset NumPy runs. Expected: 665 ms → under 60 ms.
2. Fuse the stencil (plan 9.2). The Game of Life's thirteen passes become two. Expected: 261 ms → ~100 ms, i.e. level with both.
3. Rule-dispatch cost for `fib`: the profile points at `MatchEnv` allocation and the substitution walk (plan 5.2). A small-arity fast path that matches `f[Integer]` against a literal without allocating an environment would cover the overwhelming majority of user rules. Expected: 365 ms → ~120 ms, which would put it level with Mathematica; beating CPython at function-call dispatch is a harder target and is honestly stated as such.

Items 1 and 2 are contained. Item 3 is the evaluator's own core and should be approached with a profile, not a guess.